

ONE SOURCE OF TRUTH: GENERATING NIXOS FROM NETBOX

Kierán Meinhardt / Jacek Galowicz

NixCon · Kraków / 2026-09-26 14:30:00



About Kierán

Kierán Meinhardt | github.com/kmein

Nix/NixOS consultant, test-driver maintainer | Berlin
NixOS meetup ultra | Haskell → Nix | linguist (M. A.) |



Applicative Systems | applicative.systems

Nix/NixOS consulting for the public sector and
private industry in the US and EU



About Jacek

Jacek Galowicz | @tfc

CEO of Applicative Systems/Nixcademy, MSc EE, 20+ years SWE, author (C++ & Nix Books, Heise), NixOS test driver, Nix trainer

Applicative Systems | applicative.systems

Nix/NixOS consulting for the public sector and private industry in the US and EU



Two single sources of truth?!

NetBox

```
router
eth0      10.0.0.5/24
eth0.10   vlan 10
ssh       tcp/22
```

≠

configuration.nix, 40x times

```
router
eth0      10.0.0.6/24
eth0.10   ?
allowedTCPPorts = [ 22 80 ]
```

- ▶ potential drift over time
- ▶ onboarding new machines: has to be done twice (by two teams?)
- ▶ network engineers typically don't write Nix

Prior art: NetBox's own /render-config (hacky)

```
{% extends 'base.j2' %}

{% block content %}
    system {
        host-name {{ device.name }};
        domain-name example.com;
        time-zone UTC;
        authentication-order [ password radius ];
        ntp {
            {% for server in ntp_servers %}
                server {{ server }};
            {% endfor %}
        }
    }
{% for interface in device.interfaces.all() %}
    {% include 'common/interface.j2' %}
{% endfor %}
{% endblock %}
```

Prior art: Ansible (imperative 🦠)

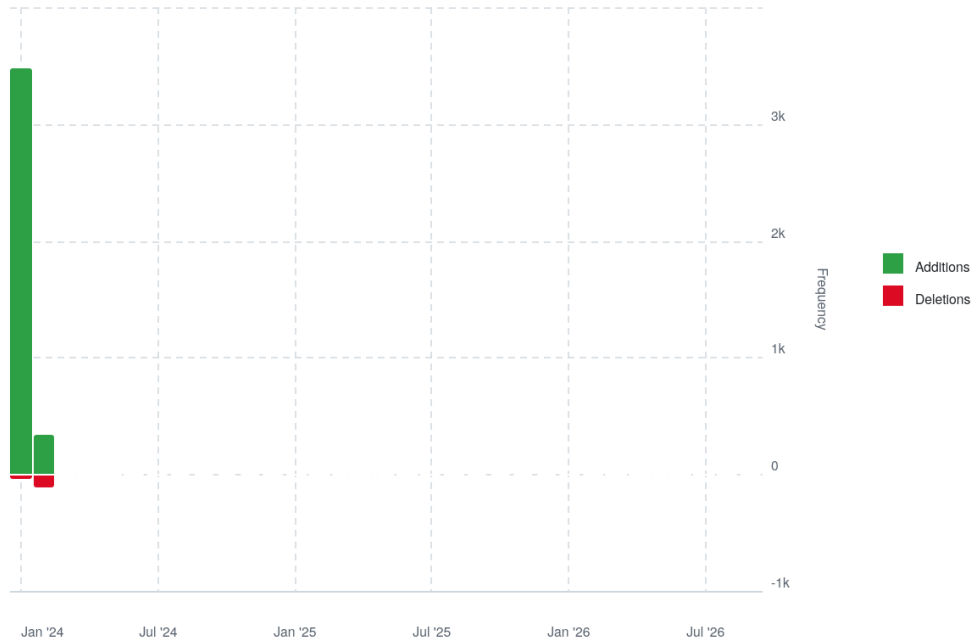
```
plugin: netbox.netbox.nb_inventory
api_endpoint: "https://netbox.example.com"
token: "0123456789abcdef0123456789abcdef01234567"
validate_certs: false
config_context: true
group_by:
  - device_roles
```

```
ansible-galaxy collection install netbox.netbox
ansible-playbook -i netbox_inventory.yml deploy.yaml
```

Prior art: NixBox (discontinued)

Code frequency

Additions and deletions per week



- ▶ <https://github.com/Janik-Haag/nixbox>
- ▶ Nix-first, writes back to NetBox
- ▶ funded by NLNet NGIO Core
- ▶ last commit May 2024

HOW ABOUT:

“NETBOX IS A FLAKE.”

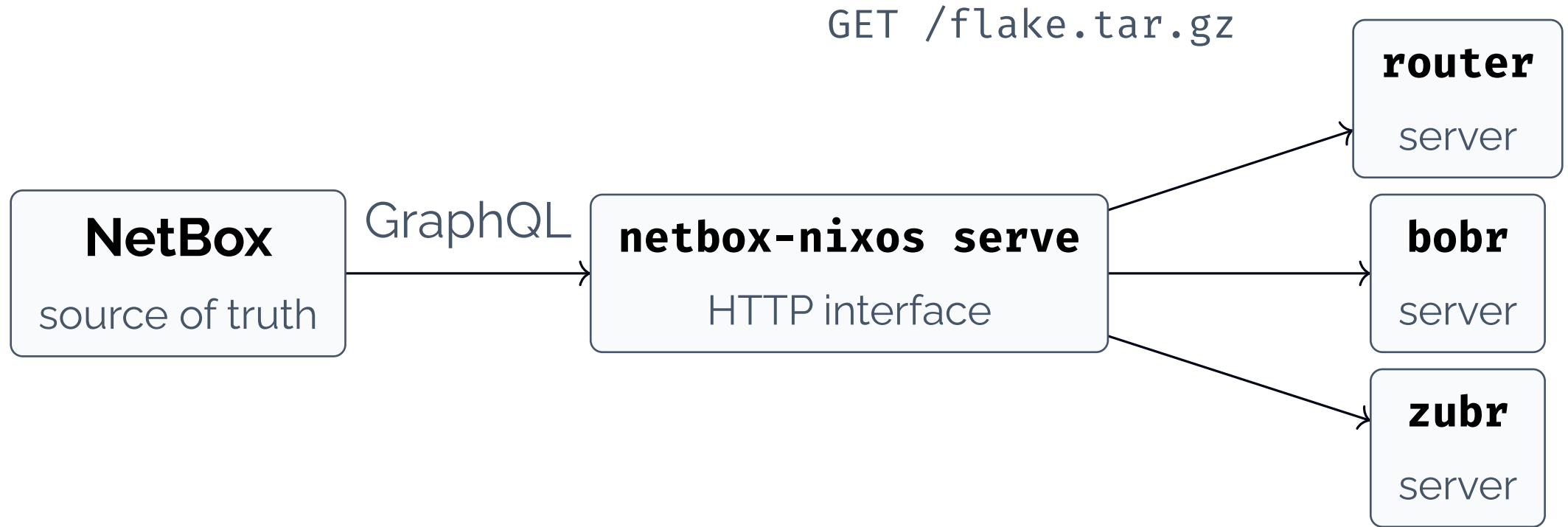
NetBox, plz give flake

```
{  
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-25.11";  
  inputs.netbox.url = "https://netbox.example.org/nix/flake.tar.gz";  
  
  outputs = inputs: {  
    nixosConfigurations.router = inputs.nixpkgs.lib.nixosSystem {  
      modules = [  
        inputs.netbox.nixosProfiles.router  
        ./hardware-configuration.nix  
      ];  
    };  
  };  
}
```

or, from the box itself:

```
[root@router:~]# nixos-rebuild switch --flake --no-write-lock-file \  
https://netbox.example.org/nix/flake.tar.gz#router
```

How it works



Input: GraphQL export from NetBox

```
{ "devices": [  
  { "name": "router", "primary_ip4": { "address": "10.0.0.5/24" },  
    "interfaces": [  
      { "name": "eth0", "type": "1000base-t", "mtu": 1500,  
        "primary_mac_address": { "mac_address": "52:54:00:12:34:56" },  
        "ip_addresses": [ { "address": "10.0.0.5/24",  
                           "dns_name": "router.example.org" } ] },  
      { "name": "eth0.10", "type": "virtual", "parent": { "name": "eth0" },  
        "untagged_vlan": { "vid": 10, "name": "lan" },  
        "ip_addresses": [ ... ] } ],  
    "services": [ { "name": "ssh", "protocol": "tcp", "ports": [ 22 ] } ],  
    "config_context": { "nixos": { "time": { "timeZone": "UTC" } } } },  
  "virtual_machines": [ ... ],  
  "netbox_version": "4.6.8"  
}
```

Output: systemd-networkd NixOS attrs

NetBox	NixOS
name	<code>networking.hostName, .domain</code>
primary IPs, dns_nameS	<code>networking.hosts</code>
physical interface	<code>systemd.network.networks</code> : MAC, MTU, addresses
virtual + parent + untagged VLAN	<code>systemd.network.netdevs</code> of kind <code>vlan</code>
services	<code>networking.firewall</code> , TCP and UDP ports
config context <code>nixos</code>	merged into the configuration as is

Netbox merging mechanism: weights

platform nixos, weight 1000

```
{ "nixos": { "networking": { "useDHCP": false } } }
```

site kraków, weight 2000

```
{ "nixos": { "time": { "timeZone": "Europe/Warsaw" },  
  "systemd": { "network": { "networks": { "10-eth0": {  
    "routes": [ { "Gateway": "10.0.0.1" } ] } } } } }
```

device router in Kraków, config_context as served

```
{ "nixos": { "networking": { "useDHCP": false },  
  "time": { "timeZone": "Europe/Warsaw" },  
  "systemd": { "network": { "networks": { "10-eth0": {  
    "routes": [ { "Gateway": "10.0.0.1" } ] } } } } }
```

Netbox merge mechanism + NixOS merge mechanism

NetBox config context

- weight decides
- by region, site, role, platform, tenant, tag
- deep merge, lighter first: 1000, then 2000
- device-local context wins

NixOS module system

- priority decides (`lib.mkOverride`, `lib.mkDefault`, `lib.mkForce`)
- NetBox config context JSON injected with priority 100
- above `mkDefault`, below `mkForce`
- merges lists, conflicts on equal scalars

Flakes on the fly

any directory with `flake.nix`, tarballed == valid Flake

```
$ nix flake show https://netbox.example.org/nix/flake.tar.gz
├── lib
│   └── data: NetBox data
├── nixosConfigurations
│   ├── bobr: NixOS configuration
│   ├── router: NixOS configuration
│   └── zubr: NixOS configuration
└── nixosProfiles
    ├── bobr: NixOS profile
    ├── router: NixOS profile
    └── zubr: NixOS profile
```

Don't boot from it (quite yet)

NetBox has

- /etc/hosts
- interfaces, VLANs, addresses
- firewall ports
- NixOS config via config context JSON

NetBox lacks

- disks, boot loader
- stateVersion
- users, secrets, software
- anything with a function, a package or a secret in it

Consume and customize

nixosProfiles

```
{
  inputs.nixpkgs.url =
    "github:NixOS/nixpkgs/nixos-25.11";
  inputs.netbox.url =
    "https://netbox.example.org/nix/flake.tar.gz";

  outputs =
    { nixpkgs, netbox, ... }:
    {
      nixosConfigurations.router =
        nixpkgs.lib.nixosSystem {
          modules = [
            netbox.nixosProfiles.router
            ./hosts/router.nix
          ];
        };
    };
}
```

nixosConfigurations

```
{
  inputs.nixpkgs.url =
    "github:NixOS/nixpkgs/nixos-25.11";
  inputs.netbox.url =
    "https://netbox.example.org/nix/flake.tar.gz";
  inputs.netbox.inputs.nixpkgs.follows = "nixpkgs";

  outputs =
    { nixpkgs, netbox, ... }:
    {
      nixosConfigurations = nixpkgs.lib.mapAttrs (
        name: system:
        system.extendModules {
          modules = [ ./hosts/${name}.nix ];
        }
      ) netbox.nixosConfigurations;
    };
}
```

nixosProfiles VS. nixosModules

for best composability, some guidelines grown from experience:

	nixosModules	nixosProfiles
encodes	mechanism	policy
example	<nixpkgs/nixos/ modules/ services/ networking/ssh/ sshd.nix>	<nixpkgs/nixos/ modules/ profiles/ perlless.nix>
import causes side-effects?	no	yes
may import other modules?	no	yes

Demo: network scenario

router, a device

eth0 · 10.0.0.5/24

eth0.10 · VLAN 10 conference

ssh · tcp/22

bobr, a server

eth0 · 10.0.0.10/24

http · tcp/80, 443

zubr, a server

eth0 · 10.0.0.11/24

dns · udp/53

context platform nixos, weight 1000

timeZone = "UTC" · openssh.enable = true

context site krakow, weight 2000

timeZone = "Europe/Warsaw" · route via 10.0.0.1

DEMO

Built for a client, currently in the process of being open-sourced

What you saw

- ▶ ~170 lines of Python stdlib
- ▶ proof of concept (do not use!)
- ▶ subset of data model, mutable URLs
- ▶ try it now: `nix run github:applicative-systems/netbox-nixos-demo#demo`

The real thing

- ▶ production Rust 
- ▶ plug your own modules in, per server
- ▶ immutable versioned URLs
- ▶ more of the data model: prefixes and routes, VRFs, tunnels, ...
- ▶ currently awaiting release, will substitute POC